BÁO CÁO PHÂN TÍCH KHOẢNG TRỐNG CỦA AI (AI GAP ANALYSIS)

Môn học: Kiểm thử phần mềm (HW02)

Sinh viên thực hiện: **Trần Vũ Quang**
Mã số sinh viên (MSSV): **23120346**
Repository GitHub: https://github.com/tinphan247/-KTPM-2026-HW02-Domain-Testing
Branch: https://github.com/tinphan247/-KTPM-2026-HW02-Domain-Testing/tree/hw02-23120346
Mục tiêu: So sánh kết quả kiểm thử do AI sinh ra so với quá trình rà soát thực tế của con người (Human Review), phân tích nguyên nhân tại sao AI bỏ sót.

---

1. Phân Tích Khoảng Trống Cho Tính Năng FR-03 (Quên & Đặt lại mật khẩu)

1.1. Những ca kiểm thử và Bug mà công cụ AI ban đầu bỏ sót

Khi yêu cầu AI sinh test case cho tính năng FR-03 chỉ dựa trên bản đặc tả tài liệu (Black-box Specification):

1. **AI bỏ sót lỗi thiếu trường "Xác nhận mật khẩu" (Confirm Password)**:
   - *Thực tế*: AI sinh ra rất nhiều ca kiểm thử hợp lệ kiểm tra xem `confirm_password` có khớp với `new_password` hay không. Tuy nhiên, AI mặc định rằng giao diện đã có sẵn ô nhập này và không nhận diện được rằng nhà phát triển giao diện đã **quên hoàn toàn việc tạo trường input `confirm_password` trên UI**.

2. **AI bỏ sót Bug lệch pha độ dài OTP giữa Backend và Đặc tả (4 số vs 6 số)**:
   - *Thực tế*: AI thiết kế các ca BVA với giả định OTP luôn là 6 chữ số ($N=5, 6, 7$). AI không lường trước được rằng Backend trong thực tế lại sinh mã OTP chỉ gồm 4 chữ số (`Math.random() * 9000`), và nhãn UI lại ghi là "Mã OTP (4 số)".

3. **AI bỏ sót lỗi Logic Regex mật khẩu (yêu cầu khoảng trắng thay vì ký tự đặc biệt)**:
   - *Thực tế*: AI tự động thiết kế ca kiểm thử mật khẩu hợp lệ là `Eshop@2026` và kỳ vọng hệ thống sẽ chấp nhận. Tuy nhiên, khi chạy thực tế trên code thật, hệ thống lại từ chối mật khẩu này vì lập trình viên đã viết sai regex thành `(?=.*\s)` (đòi hỏi khoảng trắng). Con người phải trực tiếp kiểm tra mã nguồn (`ForgotPassword.jsx`) hoặc test thực tế mới phát hiện ra bug logic nghiêm trọng này.

4. **AI HOÀN TOÀN BỎ SÓT LỖI TƯƠNG TÁC THỰC TẾ: Browser Autofill Misdirection (BUG-FR03-08)**:
   - *Phát hiện của Sinh viên*: Khi người dùng test trên trình duyệt thực tế có lưu tài khoản, do form thiếu thuộc tính `autocomplete="off"` , trình duyệt tự động điền địa chỉ email cá nhân (`tvquang.working@gmail.com`) vào ô "Mã OTP" và điền mật khẩu lưu sẵn vào ô "Mật khẩu mới"!

- *Lý do AI bỏ sót*: AI chỉ chạy trong môi trường giả lập (Sandbox/Headless) không có hồ sơ người dùng thực và không có mật khẩu lưu sẵn. AI không trải nghiệm được hành vi thực tế của cơ chế Autofill Heuristics trên Chrome/Edge. Đây là minh chứng đắt giá nhất cho vai trò không thể thay thế của **Human Review** trong triết lý AI-First của bài tập này!

1.2. Phân Tích Nguyên Nhân (Root Cause Analysis)

Theo yêu cầu mục 6 của đề bài, các nguyên nhân khiến AI bỏ sót các vấn đề trên bao gồm:

1. **Giới hạn của Prompt và Phạm vi Ngữ cảnh (Context Limitations)**:
   - Prompt ban đầu thường chỉ cung cấp tài liệu đặc tả chức năng (SRS - những gì hệ thống *nên làm*). AI không được cấp toàn bộ mã nguồn của từng file giao diện React và Backend, dẫn đến việc AI thiết kế test case theo "kịch bản lý tưởng" (Ideal World) thay vì phản ánh "thực tế triển khai" (As-built SUT).
2. **Thiên lệch giả định hệ thống tuân thủ (Compliance Bias)**:
   - Các mô hình LLM có xu hướng ngầm định rằng các thành phần cơ bản của form (như trường Confirm Password) đã được cài đặt đầy đủ theo đặc tả. AI không có trực giác nghi ngờ những lỗi cơ bản mang tính "quên cài đặt" của lập trình viên.
3. **Thiếu khả năng thực thi động (Lack of Dynamic Execution)**:
   - AI dạng văn bản (LLM) chỉ thực hiện phân tích tĩnh trên lý thuyết. AI không thể trực tiếp click chuột vào giao diện, không nhìn thấy thông báo Alert bật lên, và không tương tác trực tiếp với API runtime trừ khi được kết nối vào hệ thống kiểm thử tự động chuyên sâu.

1.3. Biện Pháp Khắc Phục Của Con Người (Human Review Remediation)

- **Can thiệp thủ công**: Sinh viên trực tiếp khởi chạy SUT, mở mã nguồn `frontend-web/src/pages/ForgotPassword.jsx` và `backend/server.js` để đối chiếu từng dòng lệnh.
- **Bổ sung Test Cases**: Bổ sung các test case kiểm tra hành vi thực tế của OTP 4 số, xử lý lỗi Autofill của trình duyệt, và kiểm tra trường hợp mật khẩu có chứa khoảng trắng.
- **Đăng tải Bug Report**: Ghi nhận toàn bộ các lỗi thực tế kèm bằng chứng ảnh chụp UI và bằng chứng phân tích mã nguồn/API.

---

2. Phân Tích Khoảng Trống Cho Tính Năng FR-07 (Giỏ Hàng & Luồng Mua Sắm)

2.1. Bug do Con Người phát hiện mà AI hoàn toàn bỏ sót: Lỗi Bắt buộc Double-Click để Thêm vào giỏ hàng

- **Hiện tượng thực tế**: Khi người dùng vào trang chi tiết sản phẩm và bấm nút **"Thêm vào giỏ hàng"**, ở lần bấm đầu tiên hệ thống hoàn toàn **im lặng, không có bất kỳ phản hồi nào** và sản phẩm không hề

được đưa vào giỏ hàng! Người dùng bắt buộc phải **nhấp đúp chuột (Double-Click)** thì sản phẩm mới được thêm vào giỏ.

- **Mã nguồn gây lỗi tại [ProductDetail.jsx:22-25]**:

```
const handleAddToCart = () => {
  if (clickCount === 0) {
    setClickCount(1);
    return; // Không làm gì cả ở lần đầu tiên
  }
  addToCart(product, parseInt(quantity));
  setAdded(true);
  setClickCount(0);
};
```

Lập trình viên đã cố tình cài đặt logic bẫy: nếu `clickCount === 0` thì chỉ tăng biến đếm lên 1 và `return`, bỏ qua hoàn toàn thao tác thêm vào giỏ hàng!

2.2. Tại sao công cụ AI ban đầu hoàn toàn bỏ sót Bug này?

1. **AI chỉ kiểm thử theo kịch bản lý thuyết của Đặc tả (Specification Idealism)**:
   - Bản đặc tả FR-07 và FR-06 chỉ mô tả: "*Nút Thêm vào giỏ hàng — sau khi bấm hiển thị phản hồi trực quan và thêm vào giỏ*". AI khi sinh ca kiểm thử chỉ tạo ra ca kiểm thử: "*Bấm nút Thêm vào giỏ hàng $\rightarrow$ Kỳ vọng giỏ hàng có sản phẩm*". AI ngầm định rằng sự kiện `onClick` luôn gọi hàm thêm giỏ hàng ngay lập tức.
2. **AI dạng Script/API không có cảm nhận về trải nghiệm tương tác (Tactile Interaction Absence)**:
   - Khi AI viết script tự động hóa hoặc test qua API backend (`POST /api/cart`), các hàm được kích hoạt trực tiếp từ mã lệnh và luôn trả về thành công ngay trong 1 lần gọi.
   - AI không thực sự "cầm chuột nhấp" vào màn hình như một người dùng thật, không trải qua cảm giác hụt hẫng khi bấm một nút mà màn hình không phản ứng.
3. **Giá trị cốt lõi của Human Review**:
   - Chỉ khi sinh viên trực tiếp mở trình duyệt web `http://localhost:5173`, đóng vai trò là khách hàng thực tế và click thử vào nút "Thêm vào giỏ hàng", người kiểm thử mới phát hiện ra sự bất thường: "*Tại sao tôi bấm một lần mà giỏ hàng vẫn là 0?*".
   - Đây là minh chứng rõ nét tiếp theo khẳng định: **Kiểm thử tự động hóa bằng AI chỉ kiểm tra được tính đúng đắn của logic dữ liệu (Correctness), còn trải nghiệm thao tác người dùng (Usability & User Behavior) bắt buộc phải có sự tham gia của con người (Human-in-the-loop)**.

2.3. Các Bug logic khác trong FR-07 mà AI dễ bỏ sót nếu không soi mã nguồn

1. **Bug nhân bản dòng khi thêm sản phẩm trùng (`CartContext.jsx:9`)**:
   - AI khi thiết kế test case độc lập thường chỉ kiểm thử thêm 1 sản phẩm một lần. Khi con người thao tác thực tế mua thêm chính sản phẩm đó lần 2 mới phát hiện bảng giỏ hàng sinh ra 2 dòng riêng biệt thay vì cộng dồn số lượng.
2. **Bug nút Xóa thiếu hộp thoại xác nhận (`Cart.jsx:51`)**:

- AI thường chỉ kiểm tra tính năng xóa có làm biến mất sản phẩm khỏi giỏ hay không, dễ dàng bỏ qua việc kiểm tra xem có cửa sổ pop-up/dialog xác nhận bật lên hay không.

---

3. Phân Tích Khoảng Trống Cho Tính Năng FR-15 (Quản Lý Sản Phẩm - Admin CRUD)

3.1. Các lỗi mà AI Black-Box bỏ sót và Con Người phát hiện:

1. **Lỗi Mass UI Overwrite khi Sửa sản phẩm (`fakeMassUpdatedProducts` tại `frontend-admin/src/App.jsx:110`)**:
   - *Đặc tả*: Khi Admin bấm Sửa một sản phẩm, chỉ sản phẩm đó thay đổi thông tin.
   - *Thực tế*: Lập trình viên cố tình viết hàm `products.map(p => ({ ...p, name: productForm.name }))`, khiến cho việc sửa 1 sản phẩm làm **toàn bộ danh sách sản phẩm trên màn hình bị đổi tên theo**!
   - *Lý do AI bỏ sót*: AI phân tích từng test case độc lập ("Sửa sản phẩm X thành Y $\rightarrow$ Kỳ vọng sản phẩm X có tên Y"). AI không kiểm tra "hiệu ứng lây lan ngoài ý muốn" (Side Effects) lên các phần tử khác trong mảng state React nếu không trực tiếp nhìn thấy toàn bộ bảng danh sách sau khi submit.
2. **Lỗi API trả về HTTP 200 OK kèm `{}` khi ID không tồn tại (`server.js:161`)**:
   - *Thực tế*: Truy vấn sản phẩm không tồn tại (`GET /api/products/99999`) trả về HTTP 200 thay vì 404 Not Found, dẫn đến sập trắng trang ở Frontend.
   - *Lý do AI bỏ sót*: AI thường kiểm tra mã trạng thái HTTP theo thói quen: nếu 200 thì coi là Pass, mà không kiểm tra xem mã 200 này có phù hợp với chuẩn RESTful hay không.
3. **Lỗ hổng Broken Access Control hoàn toàn không cần Token (`server.js:167-196`)**:
   - Các API thêm/sửa/xóa sản phẩm không hề có middleware `authenticateToken`, cho phép bất kỳ ai cũng có thể xóa sạch CSDL.

---

4. Tổng Kết Giá Trị Của Human Review Trong Quy Trình AI-First

Qua quá trình thực hiện bài tập trên các tính năng thực tế, sự khác biệt giữa năng lực kiểm thử của công cụ AI và người kiểm thử con người được tổng kết như sau:

Khía Cạnh Kiểm Thử	Năng Lực Của AI	Năng Lực Của Con Người (Human Tester)
Bao phủ lý thuyết (Domain & BVA)	Cực nhanh, chính xác theo công thức toán học	Mất nhiều thời gian tính toán thủ công
Nhận diện lỗi thiếu sót UI (Missing Elements)	Thường giả định form đã có đủ trường	Nhìn bằng mắt là phát hiện ngay ô nhập bị thiếu (như Confirm Password)
Trải nghiệm tương tác (Double-click, Autofill)	Không cảm nhận được	Cảm nhận ngay sự bất thường khi bấm nút không ăn hoặc bị điền nhầm thông tin

Khía Cạnh Kiểm Thử	Năng Lực Của AI	Năng Lực Của Con Người (Human Tester)
Bẫy logic mã nguồn (Code Traps)	Bỏ sót nếu chỉ đọc đặc tả	Đọc trực tiếp code và phát hiện hàm cố tình chặn ở lần bấm đầu
Hiệu ứng phụ dữ liệu (Side Effects)	Thường kiểm tra đơn lẻ từng mục	Nhìn toàn cục bảng dữ liệu để bắt lỗi đổi tên hàng loạt

> **Bài học rút ra**: Mô hình hiệu quả nhất luôn là **AI-First with Human Review** — mình tận dụng AI để sinh nhanh khung kiểm thử và các ca biên lý thuyết, nhưng chính con người phải trực tiếp thao tác trên ứng dụng, giữ tư duy hoài nghi và phản biện để bắt được những lỗi tương tác và bẫy logic ngầm trong mã nguồn.